

React.js Comprehensive Interview Notes

Full Stack Developer Preparation

Contents

1	Core Fundamentals	2
1.1	Virtual DOM & Reconciliation	2
1.2	JSX (JavaScript XML)	2
1.3	Props vs. State	2
2	React Hooks	3
2.1	useState	3
2.2	useEffect	3
2.3	useRef	3
3	State Management & Context API	4
4	Routing (React Router)	4
5	Performance Optimization	5
5.1	useMemo	5
5.2	useCallback	5

1 Core Fundamentals

1.1 Virtual DOM & Reconciliation

The Virtual DOM is a lightweight, in-memory representation of the actual Real DOM. React uses it to optimize rendering.

- **Diffing:** When state changes, React creates a new Virtual DOM and compares it against the old one.
- **Reconciliation:** React calculates the most efficient way to update the Real DOM, applying only the necessary changes.

1.2 JSX (JavaScript XML)

JSX allows you to write HTML-like syntax directly inside JavaScript. It is compiled by tools like Babel into `React.createElement()` calls.

- Must return a single parent element or a Fragment (`<>...</>`).
- Uses `className` instead of `class`.

1.3 Props vs. State

- **Props (Properties):** Read-only data passed from a parent component down to a child.
- **State:** Mutable data managed internally within a specific component.

```
1 // Example of Props and State
2 import React, { useState } from 'react';
3
4 function ChildComponent({ title }) { // Receiving props
5   return <h2>{title}</h2>;
6 }
7
8 export default function ParentComponent() {
9   const [count, setCount] = useState(0); // Internal state
10
11   return (
12     <>
13       <ChildComponent title="Counter App" />
14       <button onClick={() => setCount(count + 1)}>
15         Count: {count}
16       </button>
17     </>
18   );
19 }
```

2 React Hooks

2.1 useState

Used to manage local memory in functional components. Updating state triggers a re-render.

```
1 const [user, setUser] = useState({ name: 'Alice', age: 25 });
2
3 // Updating an object requires spreading the previous state
4 const updateAge = () => {
5   setUser(prev => ({ ...prev, age: prev.age + 1 }));
6 };
```

2.2 useEffect

Handles side effects like API fetching, subscriptions, and manual DOM manipulation.

- `[]`: Runs only once on mount.
- `[dependency]`: Runs on mount and when the dependency changes.
- **Cleanup function**: Returned from the effect to clear timers or abort fetches before the component unmounts.

```
1 useEffect(() => {
2   const controller = new AbortController();
3
4   fetch('https://api.example.com/data', { signal: controller.signal
5     })
6     .then(res => res.json())
7     .then(data => console.log(data))
8     .catch(err => console.error(err));
9
10  // Cleanup function
11  return () => controller.abort();
12 }, []);
```

2.3 useRef

Stores a mutable value that does *not* trigger a re-render when changed, or accesses a DOM element directly.

```
1 const inputRef = useRef(null);
2
3 const focusInput = () => {
4   inputRef.current.focus(); // Directly touching the DOM
5 };
```

3 State Management & Context API

The Context API prevents **Prop Drilling** (passing props through multiple components that do not need them) by providing global state.

```
1 import React, { createContext, useContext, useState } from 'react';
2
3 const ThemeContext = createContext();
4
5 export function ThemeProvider({ children }) {
6   const [theme, setTheme] = useState('light');
7   return (
8     <ThemeContext.Provider value={{ theme, setTheme }}>
9       {children}
10    </ThemeContext.Provider>
11  );
12 }
13
14 // Consuming the context
15 function Profile() {
16   const { theme } = useContext(ThemeContext);
17   return <div className={`theme-${theme}`}>Profile</div>;
18 }
```

4 Routing (React Router)

Transforms the app into a Single Page Application (SPA) by intercepting URL changes and swapping components without full page reloads.

```
1 import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
2
3 function App() {
4   return (
5     <BrowserRouter>
6       <nav>
7         <Link to="/">Home</Link>
8         <Link to="/dashboard">Dashboard</Link>
9       </nav>
10      <Routes>
11        <Route path="/" element={<Home />} />
12        <Route path="/dashboard" element={<Dashboard />} />
13      </Routes>
14    </BrowserRouter>
15  );
16 }
```

5 Performance Optimization

5.1 useMemo

Caches an expensive **value** so it does not recalculate on every render.

```
1 const sortedData = useMemo(() => {  
2   return heavySortingAlgorithm(data);  
3 }, [data]); // Only recalculates when 'data' changes
```

5.2 useCallback

Caches a **function definition** to prevent child components wrapped in `React.memo` from re-rendering unnecessarily.

```
1 const handleDelete = useCallback((id) => {  
2   setItems(prev => prev.filter(item => item.id !== id));  
3 }, []); // Function reference remains stable across renders
```